1. Write a simple C program tasklist_c.c that allows you to add up to 5 tasks (as strings) to a global array and print all tasks using a for loop.

File name: tasklist_c.c

This program uses a global array to store up to 5 tasks and a for loop to display them.

```
#include <stdio.h>

char tasks[5][100];
int count = 0;

int main()
{
   int i;

   printf("Enter 5 tasks:\n");

   for (i = 0; i < 5; i++)
   {
      printf("Enter task %d: ", i + 1);
      scanf(" %[^\n]", tasks[i]);
      count++;
   }

   printf("\nTask List:\n");

   for (i = 0; i < count; i++)
   {
```

```
    printf("%d. %s\n", i + 1, tasks[i]);
  }

  return 0;
}
```

**2. Modify your tasklist_c.c to add a function markTaskDone(int index) that sets the selected task to 'DONE' in the array, then print the updated list.

Hint: Use a separate status array or append ' - DONE' to the task string.**

File name: tasklist_c.c

This program uses a separate status array and a markTaskDone(int index) function.

```
#include <stdio.h>

char tasks[5][100];
int status[5] = {0};
int count = 0;

void markTaskDone(int index)
{
  if (index >= 0 && index < count)
  {
    status[index] = 1;
  }
  else
  {
    printf("Invalid task number!\n");
```

```
    }
}

int main()
{
    int i, choice;

    printf("Enter 5 tasks:\n");

    for (i = 0; i < 5; i++)
    {
        printf("Enter task %d: ", i + 1);
        scanf(" %[^\n]", tasks[i]);
        count++;
    }

    printf("\nEnter task number to mark as DONE: ");
    scanf("%d", &choice);

    markTaskDone(choice - 1);

    printf("\nUpdated Task List:\n");

    for (i = 0; i < count; i++)
    {
        if (status[i] == 1)
            printf("%d. %s - DONE\n", i + 1, tasks[i]);
        else
```

```
        printf("%d. %s - PENDING\n", i + 1, tasks[i]);
    }

    return 0;
}
```

3. Create a Task class in your favorite OOP language (Java, Python, or C++) with properties: title and isDone. Add a method markDone() that sets isDone to true, and a method display() that prints the task with its status.

File name: Task.java

The Task class has two properties, title and isDone, and two methods, markDone() and display().

```
class Task
{
    String title;
    boolean isDone;

    Task(String title)
    {
        this.title = title;
        this.isDone = false;
    }

    void markDone()
    {
        isDone = true;
    }
```

```
  void display()
  {
    if (isDone)
      System.out.println(title + " - DONE");
    else
      System.out.println(title + " - PENDING");
  }

  public static void main(String[] args)
  {
    Task t1 = new Task("Complete homework");
    Task t2 = new Task("Read a book");

    t1.markDone();

    t1.display();
    t2.display();
  }
}
```

Output:

Complete homework - DONE

Read a book - PENDING

4. Build a simple TaskList class/object that stores multiple Task objects and provides addTask(title), markTaskDone(index), and showTasks() methods. Demonstrate adding 3 tasks, marking one as done, and displaying all tasks with their statuses.

File name: TaskList.java

This program stores multiple Task objects and provides the required methods: addTask(title), markTaskDone(index), and showTasks().

```
import java.util.ArrayList;

class Task
{
   String title;
   boolean isDone;

   Task(String title)
   {
      this.title = title;
      isDone = false;
   }

   void markDone()
   {
      isDone = true;
   }

   void display()
   {
      if (isDone)
         System.out.println(title + " - DONE");
      else
         System.out.println(title + " - PENDING");
   }
```

```
}

class TaskList
{
   ArrayList<Task> tasks = new ArrayList<>();

   void addTask(String title)
   {
      tasks.add(new Task(title));
   }

   void markTaskDone(int index)
   {
      if (index >= 0 && index < tasks.size())
         tasks.get(index).markDone();
      else
         System.out.println("Invalid task index!");
   }

   void showTasks()
   {
      for (int i = 0; i < tasks.size(); i++)
      {
         System.out.print((i + 1) + ". ");
         tasks.get(i).display();
      }
   }
```

```
  public static void main(String[] args)
  {
    TaskList list = new TaskList();


    list.addTask("Complete homework");
    list.addTask("Read a book");
    list.addTask("Practice C programming");


    list.markTaskDone(1);


    System.out.println("My Task List:");
    list.showTasks();
  }
}
```

Output:

My Task List:

1. Complete homework - PENDING

2. Read a book - DONE

3. Practice C programming - PENDING

5. Compare your procedural C version and OOP TaskList version: List 3 problems you faced in the C version that were solved by using classes and methods in the OOP version.

No.	Problems in C	How OOP Solves Them
1	Task names and statuses are stored separately.	A Task object stores the title and status together.

2	The array has a fixed limit of 5 tasks.	ArrayList allows a variable number of tasks.
3	Managing task operations requires separate functions and careful indexing.	Classes and methods organize the code and make it easier to maintain.